EXPERIMENT 6: MISSIONARIES AND CANNIBALS' PROBLEM

AIM

To develop a Python program that solves the Missionaries and Cannibals Problem using Breadth-First Search (BFS) to safely transport all missionaries and cannibals across the river.

OBJECTIVE

- To understand the Missionaries and Cannibals problem in Artificial Intelligence.
- To apply the Breadth-First Search (BFS) algorithm.
- To transport all missionaries and cannibals safely across the river.
- To ensure that missionaries are never outnumbered by cannibals on either river bank.

ALGORITHM

Step 1: Start.

Step 2: Initialize the starting state as **(3 Missionaries, 3 Cannibals, Left Bank)**.

Step 3: Generate all valid boat moves (1 or 2 people).

Step 4: Check whether the new state is valid:

- Missionaries are not outnumbered by cannibals on either bank.
- The number of missionaries and cannibals remains within valid limits.

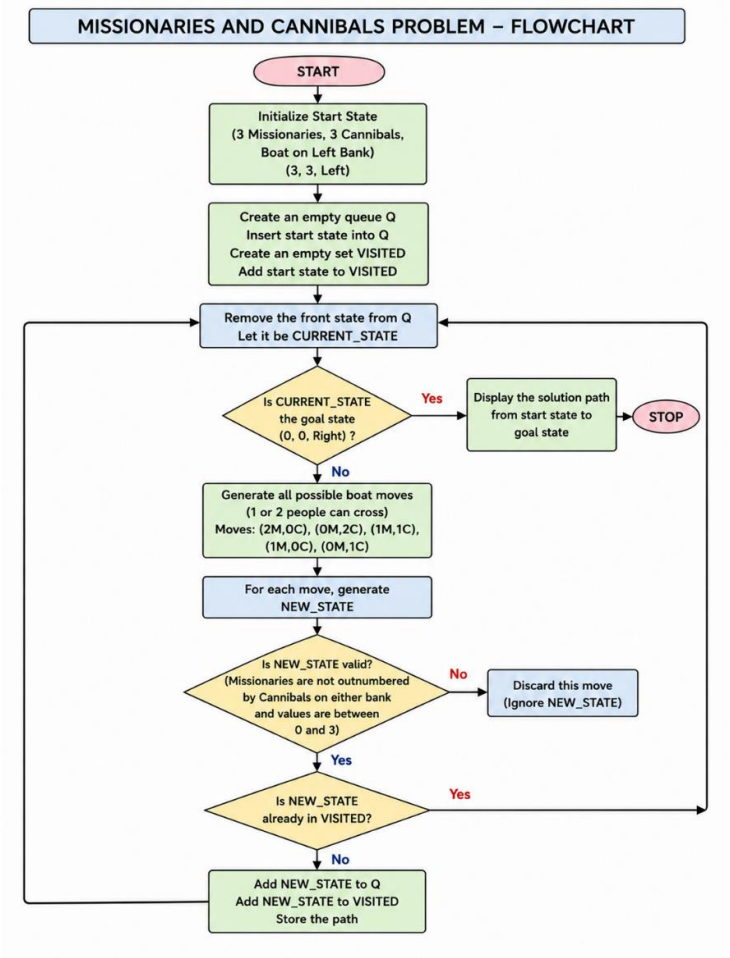
Step 5: If the state is valid and not visited, add it to the queue.

Step 6: Repeat the process until the goal state **(0 Missionaries, 0 Cannibals, Right Bank)** is reached.

Step 7: Display the solution path.

Step 8: Stop.

FLOW CHART



SOURCE CODE

```
from collections import deque


def valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False
    if (m > 0 and m < c):
        return False
    if ((3 - m) > 0 and (3 - m) < (3 - c)):
        return False
    return True
```

```
def bfs():
    start = (3, 3, 0)      # (Missionaries, Cannibals, Boat Side: 0=Left,1=Right)
    goal = (0, 0, 1)

    moves = [(2,0),(0,2),(1,1),(1,0),(0,1)]

    queue = deque([(start,[start])])
    visited = set()

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path

        if state in visited:
            continue

        visited.add(state)

        m, c, boat = state

        for dm, dc in moves:
            if boat == 0:
                new = (m-dm, c-dc, 1)
            else:
```

```
            new = (m+dm, c+dc, 0)

        if valid(new[0], new[1]):
            queue.append((new, path+[new]))

    return None

solution = bfs()

if solution:
    print("Solution Path:")
    for step in solution:
        print(step)
else:
    print("No Solution Found")
```

OUTPUT

```
Solution Path:
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)
```

RESULT

The Python program successfully solves the Missionaries and Cannibals Problem using the Breadth-First Search (BFS) algorithm and finds a valid sequence of moves to safely transport all missionaries and cannibals across the river.